

Making OpenAPI Documentation Agent-Ready: Detecting Documentation and REST Smells with a Multi-Agent LLM System

Rayfran Rocha Lima

rayfran.lima@sidia.com
Sidia Institute of Technology
Manaus, AM, Brazil

Davi G. Assuncao Pinheiro

davi.pinheiro-e@sidia.com
Sidia Institute of Technology
Manaus, AM, Brazil

Thiago Medeiros de Menezes

thiago.menezes@sidia.com
Sidia Institute of Technology
Manaus, AM, Brazil

Abstract

The growing adoption of AI agents and the Model Context Protocol (MCP) has motivated organizations to expose existing REST APIs as agent-consumable tools. In our industrial context, this initiative targeted an ecosystem of 16 production APIs comprising approximately 600 endpoints. Although these APIs were stable and widely used within a microservice architecture, early proof-of-concept experiments revealed systematic failures in task planning, tool selection, and payload construction when accessed through MCP-based agents. Rather than attributing these failures to model limitations alone, we conducted an ecosystem-scale empirical assessment of the underlying OpenAPI documentation. We developed *Hermes*, a multi-agent LLM-based system that detects documentation and REST-related smells at the endpoint level and generates explainable diagnostic reports. The large-scale evaluation identified 2,450 smells across 600 endpoints, with deficiencies present in all analyzed operations. Practitioner validation confirmed high agreement with the detected issues while also revealing contextual trade-offs in remediation decisions. The findings suggested that structural validity within microservice environments does not guarantee semantic readiness for agent-based consumption. Based on this evidence, the organization revised its adoption strategy, prioritizing selective endpoint adaptation, redefining documentation standards, and integrating automated documentation assessment into API governance workflows. This case illustrates how systematic artifact-level evaluation can function as a strategic decision-support mechanism, reducing technological risk and guiding evidence-based AI adoption in industrial software ecosystems.

CCS Concepts

• **Computing methodologies** → **Natural language processing; Artificial intelligence**; • **Software and its engineering** → **Software architectures; Software design engineering**.

Keywords

API Documentation Quality, OpenAPI, Documentation Smells, AI Agents, Industrial Case Study, Software Artifact Evaluation

ACM Reference Format:

Rayfran Rocha Lima, Davi G. Assuncao Pinheiro, and Thiago Medeiros de Menezes. 2026. Making OpenAPI Documentation Agent-Ready: Detecting Documentation and REST Smells with a Multi-Agent LLM System. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Application Programming Interfaces (APIs) play a central role in modern software ecosystems, acting as long-lived contracts that enable integration between heterogeneous systems and teams [6]. The OpenAPI Specification has become the de facto standard for documenting RESTful APIs, supporting automation activities such as client generation, testing, monitoring, and interactive documentation [17]. Consequently, the quality of OpenAPI documentation directly impacts usability, maintainability, interoperability, and system reliability [1, 20].

REST principles emphasize resource-oriented modeling, consistent use of HTTP methods, meaningful status codes, and explicit security definitions [8, 14, 19]. Violations of these principles reduce predictability and reuse, affecting both human understanding and automated processing [19]. Prior studies report that OpenAPI specifications frequently contain incomplete, ambiguous, or inconsistent descriptions, especially when generated automatically or maintained under time constraints [5, 6]. Existing validation tools predominantly focus on syntactic correctness or structural conformance, offering limited support for assessing semantic documentation quality [22].

Recent advances in large language models (LLMs) have enabled the development of AI agents capable of autonomously interacting with external systems, relying heavily on contextual artifacts such as API specifications for planning, tool selection, parameter construction, and response interpretation [7, 23]. Research highlights that agent performance is highly sensitive to the clarity, structure, and semantic richness of tool descriptions and contextual inputs [3, 10, 16].

The Model Context Protocol (MCP) has been introduced as a standardized mechanism for exposing APIs as agent-consumable tools [2], with OpenAPI specifications often serving as the primary knowledge source guiding agent reasoning. While studies demonstrate the feasibility of transforming OpenAPI specifications into tool-augmented agent environments [15, 16], documentation quality is frequently assumed rather than systematically analyzed. Failures are typically detected at runtime—during tool invocation or parameter mismatch—without addressing whether deficiencies stem from the underlying documentation [12, 15].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EASE 2026, Glasgow, Scotland, United Kingdom

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/XXXXXXX.XXXXXXX>

Documentation smells extend the notion of code smells to textual artifacts, capturing recurring deficiencies such as vague descriptions, fragmented explanations, or mixed concerns [11, 18]. Although their impact on maintainability and onboarding has been documented, systematic detection of documentation smells in OpenAPI specifications remains underexplored. Moreover, limited work explicitly connects documentation quality to the reliability of agent-based API consumption.

Motivated by these gaps, we conducted an industrial feasibility assessment in the context of building an AI-agent-based automation ecosystem integrated through MCP servers. However, early proof-of-concept (PoC) experiments revealed systematic failures when these APIs were exposed as MCP tools for agent interaction.

Rather than attributing these limitations solely to model constraints, we hypothesized—consistent with prior literature on context sensitivity in LLM-based systems—that structural and semantic deficiencies in the OpenAPI documentation could emerge as a central limiting factor. The PoC functioned as a diagnostic probe reinforcing evidence that agent behavior is strongly influenced by the semantic clarity of tool descriptions.

To investigate documentation readiness at scale, we introduced *Hermes*, a multi-agent LLM-based system designed as an evaluation instrument to detect documentation and REST-related smells at the endpoint level and generate explainable diagnostic reports. The objective was not to benchmark agent performance directly, but to assess whether the existing API ecosystem provided sufficient semantic clarity to support reliable agent-based consumption.

This paper reports the results of this empirical evaluation conducted in a real-world industrial setting. Our contributions:

- An industrial-scale empirical assessment of documentation and REST-related quality issues.
- A practitioner-validated analysis of identified deficiencies, highlighting contextual trade-offs in remediation decisions.
- Evidence demonstrating how systematic documentation assessment informed strategic AI adoption decisions and reduced projected remediation effort in an industrial software ecosystem.

By presenting this case, we illustrate how artifact-level empirical evaluation can guide technology adoption, support cost-benefit analysis, and strengthen quality governance in AI-enabled industrial software engineering contexts.

2 Background and Related Work

OpenAPI serves as a formal contract describing endpoints, parameters, request bodies, responses, and security mechanisms [4]. In microservice architectures, it frequently operates as a single source of truth for integration and orchestration tasks. However, prior empirical studies report that OpenAPI documentation often becomes semantically underspecified, particularly when automatically generated from source code or maintained under delivery pressure [6, 20].

While static analysis tools detect syntactic and structural violations in OpenAPI specifications, they provide limited support for evaluating semantic expressiveness, clarity, and coherence [22]. As a result, specifications may remain structurally valid yet insufficiently descriptive for external consumers or automated systems.

Documentation smells extend the concept of code smells to textual artifacts, capturing recurring quality deficiencies such as vague descriptions, fragmented explanations, and mixed concerns [1, 11]. While prior research has documented the influence of documentation quality on maintainability and onboarding, there is limited work explicitly connecting these deficiencies to the reliability of agent-based API consumption. Most prior work focuses on natural language documentation embedded in source code [16]. Little research operationalizes documentation smells in structured API specifications or examines their implications for automated consumption by AI agents.

Recent advances in LLM-based agents enable automated interaction with external tools and APIs [7, 23]. Frameworks such as MCP facilitate exposing APIs as agent-consumable tools [2]. Research on tool-augmented LLMs and API invocation demonstrates that agents can successfully perform complex multi-step tasks when provided with sufficiently expressive tool descriptions and contextual guidance [3, 16].

However, existing approaches predominantly evaluate execution correctness or runtime performance [10]. Documentation quality is often treated as an implicit prerequisite rather than an explicit object of empirical assessment. Failures are typically identified during execution without systematically tracing them to structural or semantic deficiencies in the specification [13, 15].

This gap is particularly relevant in industrial contexts where APIs were not originally designed for agent-based consumption. Understanding the documentation readiness of an API ecosystem—before large-scale MCP conversion—becomes a critical feasibility criterion.

Although grounded in an industrial context, similar documentation challenges—such as incomplete, ambiguous, and inconsistent specifications—are also observed in public APIs and open ecosystems. These issues often reflect implicit knowledge dependencies, hindering understanding by new consumers [5].

Taken together, prior research emphasizes runtime validation and infrastructure scalability, but offers limited support for preventive, documentation-centric evaluation of OpenAPI specifications at ecosystem scale. This study addresses that gap through an industrial case study focused on artifact-level assessment as a foundation for evidence-based AI adoption.

3 Industrial Context and Feasibility Assessment Design

3.1 Research Design and Case Study Framing

This study follows an industrial case study design in accordance with established guidelines for case study research in software engineering [21]. The objective is exploratory and evaluative: to investigate whether documentation readiness constitutes a limiting factor for large-scale AI-agent adoption in an existing industrial API ecosystem.

The case is embedded within a single industrial organization, with the OpenAPI endpoint defined as the primary unit of analysis. The ecosystem of active APIs used by the company defines the boundary of the case.

The study does not aim to establish causal inference between documentation quality and agent performance. Instead, it evaluates documentation readiness as a feasibility criterion for technology

adoption. Multiple sources of evidence were collected and analyzed, including: (i) proof-of-concept experimentation, (ii) large-scale automated smell detection, (iii) expert validation of a representative subset, and (iv) practitioner review of diagnostic reports. This triangulation increases construct validity and strengthens the robustness of the findings.

3.2 Organizational Context

The study was conducted within an industrial R&D institute operating as part of a global network focused on embedded systems development. The organization maintains a microservice-based architecture supporting project management, product configuration, integration processes, and interaction with external platforms such as issue tracking systems.

At the time of the study, the organization maintained 16 production APIs comprising approximately 600 endpoints. These APIs were actively used for front-end integration and service-to-service communication and were considered stable within their intended microservice context. However, they were not originally designed with external reuse or autonomous agent consumption in mind.

Driven by advances in AI agents and the emergence of the Model Context Protocol (MCP), the organization initiated an internal initiative to build an AI-based automation ecosystem. The strategy initially assumed that existing OpenAPI specifications could be directly converted into MCP servers, thereby exposing endpoints as agent-consumable tools.

3.3 Proof-of-Concept as Diagnostic Probe

Prior literature on LLM-based tool invocation and API-mediated agents indicates that agent performance is highly sensitive to the semantic clarity and expressiveness of tool descriptions and contextual artifacts [3, 16]. Our proof-of-concept (PoC) did not seek to establish this sensitivity as a novel phenomenon, but to assess whether similar effects manifested within our industrial ecosystem.

To this end, we conducted a controlled diagnostic experiment using locally deployed large language models integrated with MCP-generated servers derived from OpenAPI specifications.

Two MCP servers were constructed. The first was generated directly from a FastAPI application containing 10 endpoints, preserving the original OpenAPI documentation without refinement. The second server exposed the same business logic and endpoints but incorporated enriched documentation through detailed docstrings explicitly describing tool objectives, expected inputs, security requirements, output structures, and operational semantics.

Ten representative automation tasks were defined. Eight required coordination between multiple tools, and two involved single-tool interaction. These tasks reflected realistic project management scenarios.

When executing tasks through agents interacting with the first MCP server (original documentation), approximately 70% of tasks failed during the planning phase. Only 10% of expected endpoints were correctly identified, and none were successfully executed end-to-end due to payload construction or authentication errors.

In contrast, with enriched documentation, correct execution plans were generated in 90% of tasks, 80% of endpoints were correctly identified, and all identified endpoints were successfully executed.

Because both servers exposed identical business logic, the most salient experimental difference was the documentation enrichment. These findings aligned with prior research suggesting that agent reasoning is highly dependent on contextual clarity. The PoC therefore functioned as a diagnostic probe, reinforcing literature-based expectations and motivating a systematic, documentation-centric feasibility assessment at the ecosystem scale.

The PoC was designed as an exploratory diagnostic rather than a controlled experiment. Each task was executed once per configuration, without repeated runs or statistical treatment. Given the stochastic nature of LLM outputs, no explicit control of randomness (e.g., temperature or seed variation) was applied. Therefore, the results should be interpreted as indicative evidence of systemic behavior rather than definitive performance measurements. Therefore, results should not be interpreted as statistically generalizable.

3.4 Rationale for Documentation-Centric Assessment

Rather than immediately scaling MCP conversion, the organization decided to conduct a structured feasibility assessment focused on documentation readiness.

Preliminary inspection suggested that fields such as *summary* and *description* were frequently sparse, inconsistent, or oriented toward implicit team knowledge. Endpoint semantics were occasionally ambiguous, HTTP methods diverged from REST conventions, and parameter schemas relied on generic structures with limited explanatory detail.

While such characteristics did not hinder internal human developers, they posed potential risks for autonomous agents relying exclusively on OpenAPI metadata. Given the scale of the ecosystem (600 endpoints), manual inspection was impractical. A comprehensive refactoring effort was estimated at approximately 385 engineering hours, requiring a more evidence-based decision strategy.

3.5 Feasibility Assessment Design

To operationalize documentation readiness evaluation at scale, we developed *Hermes* as an assessment instrument. *Hermes* detects documentation and REST-related smells at the endpoint level and generates structured, explainable reports.

Building upon the work of Khan et al. [11], this study interprets **documentation smells** as indicators of insufficient semantic expressiveness that may hinder both human understanding and autonomous agent consumption. Four smell categories are considered: **LAZY**: Superficial or generic documentation, including short summaries, vague descriptions, undocumented parameters, or generic response messages. **BLOATED**: Excessively verbose descriptions with limited informational value. **TANGLED**: Documentation mixing unrelated concerns (e.g., business logic, security, error handling) within the same textual fragment. **FRAGMENTED**: Essential information dispersed across disconnected sections without explicit linkage.

Grounded in established REST design principles [5, 8, 14], **REST-related smells** capture recurring design inconsistencies that reduce predictability and interoperability. Five categories are considered: **PATH**: Action-oriented or inconsistently named URIs that do not represent resources. **METHOD**: Inappropriate or semantically inconsistent use of HTTP methods. **INPUT**: Weakly specified parameters or request bodies lacking semantic clarification. **RESPONSE**: Inconsistent or insufficiently described response schemas, status codes, or error handling. **SECURITY**: Missing or unclear authentication and authorization definitions.

Hermes is designed to evaluate documentation readiness of OpenAPI specifications at the endpoint level. Given the URL of an OpenAPI JSON file, the system enables interactive exploration and automated smell detection for individual API operations.

Rather than analyzing the specification as a monolithic artifact, Hermes adopts an *endpoint-centric* strategy: each operation (defined by the combination of HTTP method and path) is isolated and evaluated independently. For a selected endpoint, the system generates a reduced OpenAPI representation containing only the relevant elements (method, path, summaries, descriptions, parameters, request body, responses, schemas, and security definitions). An illustrative excerpt of the reduced OpenAPI representation used in the analysis is provided in Appendix B.

This reduction serves two purposes. First, it decreases token consumption for large language models, addressing context-window limitations. Second, it improves analytical focus by removing unrelated endpoints and documentation fragments that could introduce ambiguity. The resulting artifact preserves all semantic information necessary for smell detection while enabling scalable analysis across large ecosystems.

Hermes follows a multi-agent architecture. A central Smell Detector Agent orchestrates the workflow and consolidates results, while specialized agents analyze specific smell categories. Documentation Smell Agents evaluate textual deficiencies (e.g., LAZY, BLOATED, TANGLED, FRAGMENTED), and REST Smell Agents evaluate design-related inconsistencies (e.g., PATH, METHOD, INPUT, RESPONSE, SECURITY). The prompt template used by the specialized agents, including the definition and classification rules for the Lazy documentation smell, is provided in Appendix A.

Each specialized agent operates based on a structured prompt that encodes the definition of the target smell, along with classification criteria and illustrative examples following a few-shot prompting strategy. All agents receive the same reduced endpoint-specific OpenAPI representation but analyze it from distinct perspectives according to their assigned smell category. The prompts guide agents to produce structured, explainable outputs, including justification of detected issues and suggested improvements, supporting consistency and interpretability across analyses.

All agents receive the same reduced endpoint representation and apply structured prompting based on the smell taxonomy. Their outputs are aggregated into a unified, explainable diagnostic report. This modular design supports extensibility and enables consistent, scalable documentation assessment across the entire API ecosystem.

Since Hermes relies on large language models (LLMs) for semantic smell detection, selecting a model with reliable multi-label classification performance was necessary before conducting the large-scale ecosystem assessment.

A representative subset of 60 endpoints (10% of the 600 production endpoints) was randomly selected to construct a gold standard dataset. Two domain experts manually annotated documentation and REST-related smells according to the taxonomy defined in Section 3. Disagreements were resolved through discussion to produce consolidated reference labels.

Smell detection was formulated as a multi-label classification task, and model performance was evaluated using standard agreement metrics, including Jaccard similarity, F1 scores (micro and macro), and Hamming loss. These metrics collectively capture overlap with expert annotations, balance across frequent and infrequent smell categories, and prediction error rate.

Seven LLMs were evaluated under identical prompting conditions to ensure comparability. As shown in Table 1, *gpt-oss:120b* achieved the highest agreement with the expert-annotated gold standard (Jaccard = 0.85; $F1_{micro}$ = 0.92; $F1_{macro}$ = 0.73) and the lowest Hamming loss (0.07), demonstrating more stable multi-label behavior across smell categories. Cardinality difference (column Cardin. Diff) presents the deviation between predicted and actual number of labels per instance.

Table 1: Overall accuracy metrics for smell detection per LLM

Model	Jaccard	$F1_{micro}$	$F1_{macro}$	Hamming	Cardin. Diff
gpt-oss:120b	0.85	0.92	0.73	0.07	-0.53
gemma3:27b	0.53	0.70	0.50	0.37	+1.83
qwen2.5vl:72b	0.67	0.80	0.49	0.20	-0.47
deepseek-r1:70b	0.69	0.81	0.48	0.16	-1.22
llama3.2-vision:90b	0.62	0.77	0.46	0.21	-1.17
llama4:16x17b	0.51	0.68	0.44	0.29	-1.18
qwen3-coder:30b	0.53	0.70	0.38	0.25	-1.72

Given its superior alignment with expert judgment and balanced prediction behavior, *gpt-oss:120b* was selected as the operational model for the full ecosystem assessment. The purpose of this evaluation was not to benchmark LLMs broadly, but to ensure analytical reliability before applying Hermes to all 600 industrial endpoints.

This selection step increases confidence that the large-scale documentation readiness results reflect ecosystem characteristics rather than model-specific prediction bias.

The feasibility assessment comprised four coordinated activities:

- **Gold Standard Definition:** Manual annotation of a representative 10% subset (60 endpoints) by domain experts to evaluate model reliability.
- **Practitioner Evaluation:** Structured review of selected diagnostic reports by API authors and maintainers.
- **Large-Scale Technical Assessment:** Automated smell detection across all 600 endpoints.
- **Strategic Cost Estimation:** Comparative estimation of remediation effort under full-ecosystem refactoring versus selective endpoint adaptation.

Data analysis combined quantitative ecosystem-level metrics (e.g., smell frequency and distribution), multi-label classification accuracy measures, qualitative practitioner feedback, and cost comparison analysis. This combination of quantitative and qualitative

evidence supports analytical triangulation and strengthens internal validity.

3.6 Industrial Evaluation Questions

To structure the analysis, we formulated three Industrial Evaluation Questions (IEQs):

- **IEQ1 (Documentation Readiness):** To what extent does the OpenAPI ecosystem exhibit documentation and REST-related deficiencies that may compromise reliable agent-based consumption?
- **IEQ2 (Practitioner Validation):** To what extent do practitioners agree with identified deficiencies, and how do contextual constraints shape remediation decisions?
- **IEQ3 (Strategic Feasibility):** What are the feasibility and cost implications of converting the API ecosystem into MCP-based agent tools under current documentation conditions?

These questions frame the study as an industrial feasibility evaluation rather than a benchmark of agent performance, aligning the investigation with decision-making needs in technology adoption contexts.

4 Results

This section presents the findings structured around the Industrial Evaluation Questions (IEQs), emphasizing documentation readiness, practitioner validation, and strategic feasibility implications.

4.1 IEQ1 – Documentation Readiness

IEQ1 investigates the extent to which the existing OpenAPI ecosystem exhibits documentation and REST-related deficiencies that could compromise reliable agent-based consumption.

Following the model selection process described in Section 3.5, the selected LLM (*gpt-oss:120b*) was applied to the full ecosystem comprising 600 production endpoints distributed across 16 APIs. All endpoints were active, validated, and operational within the organization’s microservice architecture at the time of analysis.

Overall Smell Distribution. The automated assessment performed by *Hermes* identified a total of 2,450 documentation and REST-related smells across the 600 analyzed endpoints. At least one smell was detected in every endpoint, resulting in an average of 4.08 smells per endpoint. Rather than indicating critical design failure, this distribution suggests that documentation and design deficiencies were pervasive and systematically embedded in the ecosystem. The findings point to recurring patterns of semantic underspecification and implicit assumptions that, while manageable within human-centered workflows, may introduce structural risks under autonomous agent consumption.

Table 2 presents the frequency and percentage distribution of detected smells per category.

Response-related smells were identified in all endpoints, indicating that although response schemas were technically defined, they frequently lacked explicit semantic clarification regarding returned structures, success/error interpretation, or usage semantics. An illustrative example of a complete *Hermes* diagnostic report and its corresponding OpenAPI specification excerpt is provided in Appendices B and C.

Table 2: Distribution of detected documentation and REST-related smells.

Smell Category	Frequency	% of Endpoints
Response	600	100%
Lazy	540	90%
Input	530	88%
Security	410	68%
Path_and_Method	320	53%
Tangled	30	5%
Bloated	10	2%
Fragmented	10	2%

A recurring structural pattern observed in several endpoints involved generic DTO-based responses structured as `status, data`, where the data field could contain arbitrarily shaped JSON payloads without explicit schema constraints. While such patterns were feasible and sufficient for the original developers during implementation, they introduced semantic opacity over time. In some cases, even the original endpoint authors were unable to precisely describe the possible contents of the data field without revisiting the source code. This finding illustrates that response-related smells did not merely reflect documentation sparsity, but exposed deeper issues of implicit knowledge embedded in implementation rather than in the OpenAPI specification.

Similarly, *Lazy* (90%) and *Input* (88%) smells were highly prevalent, revealing widespread sparsity or ambiguity in endpoint summaries, descriptions, and parameter documentation.

Security-related smells (68%) suggest that authentication and authorization mechanisms, while structurally specified, often lacked operational guidance or contextual explanation. Additionally, 53% of endpoints exhibited *Path_and_Method* inconsistencies, reflecting deviations from REST design conventions such as action-oriented URIs or non-idiomatic HTTP method usage [9].

In contrast, textual excess smells such as *Bloated* (2%) and *Tangled* (5%) were comparatively rare. This distribution aligns with the minimalistic documentation style observed in the organization, where documentation tended to be sparse rather than overly verbose.

Recurring Evidence Patterns. These patterns are further illustrated through a concrete example combining specification and analysis report in Appendices B and C. Table 3 summarizes the main recurring evidence patterns observed in the reports, capturing how smells typically manifested in the analyzed OpenAPI specifications.

These evidence patterns help clarify why smell detection is more reliable for categories grounded in explicit structural elements of OpenAPI specifications (e.g., *Input*, *Response*, and *Security*) and more dependent on contextual writing quality for smells centered on natural language fields (e.g., *Lazy*).

In the studied organizational context, endpoint *summaries* and *descriptions* were often short, generic, and occasionally semantically inconsistent. Documentation was primarily produced to support internal microservice integration and relied heavily on tacit knowledge shared among teams. While such practices did not hinder human developers familiar with the system, they reduced interpretability for agents relying exclusively on specification metadata.

Table 3: Recurring evidence patterns supporting smell identification in Hermes reports.

Smell	Main recurring evidence patterns observed in Hermes reports
<i>Lazy</i>	Very short or generic <i>summary</i> (e.g., “Get data”), absent or redundant <i>description</i> , and occasional semantic mismatch between <i>summary</i> and <i>description</i> . Limited contextual explanation of the endpoint purpose and lack of illustrative examples or constraints, requiring external knowledge to interpret intent.
<i>Input</i>	Parameters specified only by type (e.g., string, integer) with little or no semantic description; ambiguous parameter names (abbreviations or internal acronyms) without explanation; missing information about valid ranges, accepted formats, or behavioral impact; required parameters without functional rationale.
<i>Response</i>	Presence of response schemas (often via \$ref) with minimal explanatory text; generic response descriptions (e.g., “Successful Response”); limited semantic clarification of payload meaning across success and error cases; insufficient guidance on how consumers should interpret returned structures.
<i>Security</i>	Security schemes technically defined (e.g., Bearer token) but lacking operational guidance (how to obtain credentials, required scopes/permissions, or access constraints); security treated as implicit organizational knowledge rather than documented usage instructions.
<i>Path_and_Method</i>	Non-idiomatic or inconsistent HTTP method usage (e.g., using POST for updates that would typically be modeled as PUT or PATCH); recurrent action-oriented URIs with verbs embedded in the path (e.g., /getUsers, /updateStatus), rather than resource-oriented naming; long or semantically opaque paths containing internal terms or acronyms without clarification.
<i>Bloated / Tangled</i> (rare)	Textual fields (<i>summary/description</i>) were generally short and low-density, reducing opportunities for excessive verbosity (<i>Bloated</i>) or mixing multiple concerns within the same documentation fragment (<i>Tangled</i>). Their scarcity is consistent with minimalistic documentation practices rather than evidence of consistently high-quality narratives.
<i>Excessive Structured</i> (absent)	The agent was configured to distinguish normative OpenAPI structure (schemas and references under <i>components</i>) from cases where excessively structured content is embedded in natural language fields. The smell would apply if class-like definitions, nested structures, or formal specifications were written inside <i>summary</i> or <i>description</i> ; however, no such cases were identified in the reports.
<i>Fragmented</i> (absent)	All referenced schemas/components were defined within the same OpenAPI document (typically under <i>components</i>), with no broken or missing references. Fragmentation would be identified if endpoints referenced schemas/components not present in the specification, resulting in incomplete documentation; this condition was not observed in the analyzed reports.

Reliability of Large-Scale Assessment. As reported in Section 3.5, the selected LLM achieved strong agreement with expert annotations on a representative subset of 60 endpoints (Jaccard = 0.85; $F1_{micro}$ = 0.92; Hamming Loss = 0.07). This level of alignment provides confidence that the large-scale ecosystem results reflect structural documentation characteristics rather than model-specific prediction bias.

Implications for Agent Readiness. From a documentation readiness perspective, these findings indicate that although the APIs were structurally valid and operational within a microservice environment, their specifications lacked the semantic richness and explicitness required for autonomous agent consumption.

Structural correctness did not imply agent-readiness. High prevalence of input, response, and description-related smells suggests that the ecosystem was optimized for human developers operating within shared organizational context, rather than for external or autonomous consumers requiring explicit and self-contained documentation cues.

Therefore, IEQ1 reveals that documentation and REST-related deficiencies were systemic across the ecosystem and represent a structural risk for reliable MCP-based agent integration under the existing documentation conditions.

4.2 IEQ2 – Practitioner Validation and Contextual Constraints

IEQ2 examines practitioner agreement with the identified deficiencies and explores how contextual constraints influence remediation

decisions. Responses were collected using a Likert-scale questionnaire and analyzed through descriptive statistics, complemented by qualitative interpretation of practitioner feedback.

Using the same diagnostic reports employed during the LLM evaluation against the gold standard (a sample of 60 endpoints), we collected feedback from 24 developers, including API authors and maintainers, through a structured questionnaire.

Overall, practitioners demonstrated high agreement (80% strongly agree, 7% agree) with smells grounded in explicit and verifiable documentation elements, particularly those related to incomplete parameter descriptions, insufficient response explanations, and missing or unclear security definitions. These issues were widely perceived as objective documentation deficiencies.

However, disagreement (30% disagree) emerged in cases where smell classification intersected with contextual trade-offs. The *Lazy* smell – characterized by overly brief or generic summaries – was frequently contested. Several practitioners initially justified concise descriptions as sufficient within stable, internally shared microservice environments. When prompted to consider external consumers or AI agents relying solely on the OpenAPI specification, participants more readily acknowledged the limitations of such documentation practices.

Similar patterns were observed for REST-related smells such as non-idiomatic method usage or action-oriented URIs (20% disagree, 3% strong disagree). While deviations from REST conventions were recognized, they were sometimes justified by legacy constraints or organizational standards (32% tried to justify). This highlights the sociotechnical nature of documentation quality: deficiencies are not merely technical artifacts but also reflections of inherited practices and implicit assumptions about API consumers.

Importantly, practitioners reported a learning effect during the evaluation process. Reviewing diagnostic reports exposed implicit documentation assumptions and increased awareness of REST design principles. This suggests that documentation assessment can function not only as a quality control mechanism but also as a reflective organizational artifact.

4.3 IEQ3 – Strategic Feasibility and Cost Implications

IEQ3 addresses the feasibility and cost implications of converting the existing API ecosystem into MCP-based agent tools under current documentation conditions. The selected 42 endpoints were directly derived from the functional requirements of the 18 defined automation scenarios, ensuring alignment between documentation assessment and practical use cases.

The initial proof-of-concept experiments revealed that approximately 70% of evaluated automation scenarios failed before producing a valid execution plan. In other cases, semantically incorrect tools were selected due to ambiguous endpoint descriptions, leading to invalid payload construction or response misinterpretation. These failures were not attributable to API instability, but to insufficient semantic cues in the OpenAPI specifications guiding agent reasoning.

Based on the documentation assessment results, converting all 600 endpoints into MCP servers without prior remediation would likely propagate these limitations at scale. A preliminary estimation indicated that fully refactoring documentation across the entire ecosystem would require approximately 385 hours of coordinated engineering effort. In contrast, prioritizing the 42 endpoints directly required for the targeted 18 automation tasks reduced the estimated effort to approximately 42 hours, representing a substantial reduction (89%) in required engineering effort.

These findings informed a strategic revision of the adoption plan. Instead of large-scale MCP conversion, the organization adopted a staged approach: (i) selective adaptation of endpoints relevant to defined automation scenarios, (ii) refinement of documentation standards aligned with agent-consumption requirements, and (iii) integration of *Hermes* into the endpoint review process as a preventive assessment mechanism.

From a feasibility standpoint, the evaluation demonstrated that the primary limiting factor was not the underlying AI models or MCP infrastructure, but the documentation readiness of the API ecosystem. The assessment therefore functioned as a risk identification mechanism, enabling evidence-based redirection of investment and preventing large-scale rework under inadequate documentation conditions.

5 Discussion

This industrial case highlights a structural lesson for organizations pursuing AI-agent integration: the readiness of underlying software artifacts can be a more decisive feasibility constraint than model capability or infrastructure maturity.

Although the studied API ecosystem was operationally stable within a microservice architecture, the transition to autonomous agent consumption exposed latent semantic fragilities. Documentation practices sufficient for human developers operating under

shared contextual knowledge proved inadequate for agents relying exclusively on explicit OpenAPI metadata. In this sense, documentation deficiencies functioned as a form of architectural debt—dormant in human-mediated workflows, yet activated under autonomous reasoning.

The proof-of-concept experiments acted as a stress test for the ecosystem. Rather than revealing API instability, they exposed misalignment between the semantic richness required for agent planning and the level of explicitness in the specifications. The primary bottleneck was not AI capability, but artifact maturity. By reframing the problem as a documentation-readiness issue, the organization avoided misdirected investment in increasingly complex agent configurations.

The large-scale assessment further demonstrated that structural validity does not imply agent-readiness. OpenAPI specifications may be syntactically correct and operational within internal environments while lacking the semantic clarity required for reliable autonomous consumption. This gap underscores the need to treat documentation not merely as a descriptive artifact, but as an executable cognitive contract shaping machine reasoning.

From a strategic perspective, the evaluation enabled an evidence-based revision of the AI adoption plan. Instead of full-ecosystem MCP conversion, the organization adopted a scenario-driven adaptation strategy focused on endpoints supporting defined automation use cases. This shift reduced projected remediation effort and mitigated the risk of propagating systemic failures. The key insight is methodological: ecosystem-level readiness assessment should precede large-scale technological rollout.

An additional outcome was the transformation of *Hermes* from a diagnostic instrument into a governance mechanism. By integrating automated documentation assessment into endpoint review workflows, the organization institutionalized preventive quality control, making documentation readiness an explicit architectural concern. This illustrates how artifact-level evaluation can influence organizational practices beyond the initial technical objective.

More broadly, this case suggests that AI-agent adoption should be understood as a sociotechnical transition rather than a purely technological upgrade. Documentation smells and REST inconsistencies were often rooted in legacy conventions, implicit assumptions, and locally optimized design habits. The introduction of agents surfaced these assumptions and enabled a reflective space for revisiting documentation standards.

For industrial environments, three practical implications emerge. First, documentation readiness should be evaluated before exposing APIs as agent-consumable tools, as structural validity may conceal latent semantic deficiencies. Second, conformance to OpenAPI or REST principles does not guarantee semantic sufficiency, since agents depend on explicit, self-contained descriptions. Third, when documentation maturity is uneven, a selective, use-case-driven adaptation strategy may be more sustainable than full-ecosystem conversion.

Ultimately, this study reinforces a central EASE principle: disciplined evaluation must accompany technological enthusiasm. Evidence-based assessment of foundational software artifacts is essential to reduce strategic risk, optimize investment decisions, and enable sustainable integration of intelligent agents.

This case demonstrates that artifact-level readiness assessment can function as a gatekeeping mechanism in AI adoption roadmaps, enabling identification of structural limitations before large-scale deployment.

Although grounded in an industrial setting, these findings are expected to generalize to public API ecosystems and open platforms, where implicit knowledge assumptions are less viable and documentation must support a broader range of consumers, including both human developers and AI agents.

6 Limitations

This study was conducted within a single industrial R&D organization operating a microservice-based ecosystem composed of internally consumed APIs. Consequently, the specific distribution, prevalence, and types of documentation deficiencies observed may differ in public API platforms, open ecosystems, or organizations with distinct documentation cultures and governance practices. Although the quantitative patterns reported here may vary across contexts, the central insight—that documentation readiness influences the feasibility of agent-based API consumption—is likely transferable to environments characterized by legacy constraints, implicit knowledge dependencies, and documentation practices optimized for human-centric integration.

The assessment relied on large language models (LLMs) to perform semantic smell detection across OpenAPI specifications. While multiple models were comparatively evaluated under standardized prompting conditions to strengthen analytical reliability, some sensitivity to model configuration, inference dynamics, and prompt interpretation may remain. Model selection was additionally constrained to those capable of operating within the organization’s internal infrastructure. This restriction avoided transmitting proprietary API specifications to external servers and ensured compliance with internal security and confidentiality policies. Although this constraint may limit comparability with externally hosted frontier models, the strong agreement observed between the selected model and expert annotations increases confidence that the reported patterns reflect ecosystem characteristics rather than isolated model behavior.

The proof-of-concept component examined a limited set of representative automation scenarios rather than exhaustively evaluating all possible agent interactions. As such, reported failure rates should be interpreted as indicative rather than definitive performance benchmarks. Similarly, remediation effort estimates (e.g., 385 versus 42 engineering hours) represent planning-level approximations derived from internal cost projections. While absolute values may vary under different organizational conditions, the relative difference between full-ecosystem refactoring and selective adaptation remains strategically informative.

Finally, the evaluation reflects a specific technological configuration of MCP tooling and LLM capabilities at the time of analysis. Advances in agent reasoning, tool orchestration, or automated schema interpretation may reduce certain documentation-related limitations in future systems. However, relying exclusively on increasingly capable models to compensate for semantically ambiguous or underspecified API contracts may introduce additional operational and governance risks. For this reason, documentation quality is

framed in this study as a structural readiness concern rather than a transient tooling limitation.

7 Conclusion

This paper reported an industrial feasibility assessment conducted prior to large-scale adoption of AI-agent-based automation through MCP integration. Although the organization maintained a stable and operational API ecosystem composed of 16 production APIs and approximately 600 endpoints, the empirical evaluation suggested that structural and semantic deficiencies in the OpenAPI documentation may have constrained its readiness for autonomous agent consumption under the evaluated conditions.

Consistent with prior work, our findings reinforce that documentation quality constitutes a foundational dependency for reliable agent-based workflows. Structural validity and operational stability within microservice environments did not imply semantic sufficiency for AI-driven tool invocation. By systematically analyzing documentation and REST-related smells at scale, and validating findings with practitioners, the organization identified structural risks that would likely have compromised large-scale MCP conversion.

Rather than abandoning the strategic direction of AI-agent integration, the evidence supported a revised adoption strategy. The organization transitioned from full-ecosystem conversion to a selective, scenario-driven approach, prioritized documentation remediation where necessary, and integrated automated documentation assessment into its API governance process. This shift reduced projected remediation effort, mitigated infrastructure risk, and embedded documentation readiness as an explicit architectural concern.

Beyond the specific industrial context, this case illustrates the broader importance of evidence-based evaluation in technology adoption decisions. In environments characterized by rapid AI innovation, disciplined assessment of underlying software artifacts remains essential to ensure that enthusiasm for new paradigms is supported by structural readiness and sustainable governance practices.

By positioning documentation quality as a prerequisite rather than a byproduct of AI-agent integration, this study contributes to raising the profile of evaluation and assessment as core practices in industrial software engineering. These findings position documentation quality not as a secondary artifact, but as a foundational prerequisite for reliable AI-driven software ecosystems.

Future work may extend this evaluation in three directions. First, longitudinal studies could measure how documentation governance practices evolve after integrating preventive assessment mechanisms. Second, controlled experiments may quantify the impact of documentation remediation on agent performance metrics. Third, replication in different industrial domains and API ecosystems would strengthen the external validity of the findings.

Acknowledgments

This paper is a result of the Research, Development & Innovation Project named COPILOTO at Sidia Institute of Science and Technology sponsored by Samsung Eletrônica da Amazônia Ltda., using resources under terms of Federal Law No. 8.387/1991, by having its disclosure and publicity in accordance with art. 39 of Decree No. 10.521/2020.

References

[1] Abdullah AH Alzahrani. 2024. Software Systems Documentation: A Systematic Review. *International Journal of Advanced Computer Science & Applications* 15, 8 (2024).

[2] Anthropic. 2024. Model Context Protocol (MCP). <https://modelcontextprotocol.io>.

[3] Jayachandu Bandlamudi, Ritwik Chaudhuri, Neelamadhav Gantayat, Sambit Ghosh, Kushal Mukherjee, Prerna Agarwal, Renuka Sindhgatta, and Sameep Mehta. 2025. A framework for testing and adapting rest apis as llm tools. *arXiv preprint arXiv:2504.15546* (2025).

[4] Sandra Casas, Diana Cruz, Graciela Vidal, and Marcela Constanzo. 2021. Uses and applications of the OpenAPI/Swagger specification: a systematic mapping of the literature. In *2021 40th International Conference of the Chilean Computer Science Society (SCCC)*. IEEE, 1–8.

[5] Michael Coblenz, Wentao Guo, Kamatchi Voozhian, and Jeffrey S. Foster. 2023. A Qualitative Study of REST API Design and Specification Practices. In *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 148–156.

[6] Leonardo da Rocha Araújo, Guillermo Rodríguez, Santiago Vidal, Claudia Marcos, and Rodrigo Pereira dos Santos. 2022. Empirical Analysis on OpenAPI Topic Exploration and Discovery to Support the Developer Community. *Computing and Informatics* 40, 6 (2022), 1345–1369. doi:10.31577/cai_2021_6_1345

[7] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[8] Roy Thomas Fielding. 2000. *Architectural Styles and the Design of Network-based Software Architectures*. Ph.D. Dissertation. University of California, Irvine.

[9] P Gowda and AN Gowda. 2024. Best Practices in REST API Design for Enhanced Scalability and Security. *Journal of Artificial Intelligence, Machine Learning and Data Science* 2, 1 (2024), 827–830.

[10] Zikang Guo, Benfeng Xu, Chiwei Zhu, Wentao Hong, Xiaorui Wang, and Zhen-dong Mao. 2025. MCP-AgentBench: Evaluating Real-World Language Agent Performance with MCP-Mediated Tools. *arXiv preprint arXiv:2509.09734* (2025).

[11] Junaed Younus Khan, Md. Tawkat Islam Khondaker, Gias Uddin, and Anindya Iqbal. 2021. Automatic Detection of Five API Documentation Smells: Practitioners' Perspectives. *arXiv preprint arXiv:2102.08486* (2021). <https://arxiv.org/abs/2102.08486>

[12] Zhihao Li, Kun Li, Boyang Ma, Minghui Xu, Yue Zhang, and Xiuzhen Cheng. 2025. We urgently need privilege management in mcp: A measurement of api usage in mcp ecosystems. *arXiv preprint arXiv:2507.06250* (2025).

[13] Rayfran Rocha Lima, Irineu Evangelista Cruz de Brito, Jardel da Cunha Nascimento, and Elias Nascimento Nogueira. 2025. Empowering Conversational Systems Through AI Agents Specialized in Reusable APIs Across Software Ecosystems. In *2025 IEEE 37th International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE Computer Society, Los Alamitos, CA, USA, 1316–1323. doi:10.1109/ICTAI66417.2025.00191

[14] Mark Masse. 2011. *REST API design rulebook: designing consistent RESTful web service interfaces*. " O'Reilly Media, Inc".

[15] Meriem Mastouri, Emna Ksontini, and Wael Kessentini. 2025. Making rest apis agent-ready: From openapi to mcp servers for tool-augmented llms. *arXiv preprint arXiv:2507.16044* (2025).

[16] Xinyi Ni, Haonan Jian, Qiuyang Wang, Vedanshi Chetan Shah, and Pengyu Hong. 2025. Doc2Agent: Scalable Generation of Tool-Using Agents from API Documentation. *arXiv preprint arXiv:2506.19998* (2025).

[17] OpenAPI Initiative. 2024. OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>.

[18] Annibale Panichella, Sebastiano Panichella, Gordon Fraser, Anand Ashok Sawant, and Vincent J Hellendoorn. 2022. Test smells 20 years later: detectability, validity, and reliability. *Empirical Software Engineering* 27, 7 (2022), 170.

[19] Cesare Pautasso, Olaf Zimmermann, and Frank Leymann. 2014. RESTful Web Services: Principles, Patterns, Emerging Technologies. *IEEE Software* 31, 3 (2014), 54–61.

[20] Martin P. Robillard. 2017. What makes APIs hard to learn? Answers from developers. *IEEE Software* 34, 6 (2017), 27–34.

[21] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164. doi:10.1007/s10664-008-9102-8

[22] Theo Theunissen, Uwe van Heesch, and Paris Avgeriou. 2022. A mapping study on documentation in Continuous Software Development. *Information and Software Technology* 142 (2022), 106733. doi:10.1016/j.infsof.2021.106733

[23] Jules White et al. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. *arXiv preprint arXiv:2302.11382* (2023).

APPENDIX

A Example of Prompt Used by Specialized Agents

Box A.1: Prompt Template for the Lazy Documentation Smell Agent

Instructions for the Specialist Agent

Role:
You are an expert in identifying "Lazy" documentation smells in API documentation.

Smell Definition (Lazy):
An API method should be classified as Lazy when its documentation is incomplete, vague, or generic, failing to provide essential information about the method's purpose, behavior, parameters, return values, or possible exceptions. This includes missing examples or unclear explanations that hinder correct understanding and usage.

This smell typically occurs when:
{lazy_examples}

Task:
Analyze the complete OpenAPI method definition provided below and identify whether the endpoint exhibits Lazy documentation.

OpenAPI Specification:
{openapi_json}

For each endpoint identified, you must classify the smell, justify the classification, and propose improvement actions.

Classification Rules:

- Analyze ONLY the method summary and description.
- Return ONLY a valid JSON object containing the endpoints with Lazy smells.
- Endpoint format: "GET:/users", "POST:/orders", etc.
- If no Lazy smell is found, return an empty JSON object.
- Do NOT include any text before or after the JSON output.

Explanation and Improvement Rules:

- Analyze ONLY the method summary and description.
- Provide explanations with a minimum length of 120 characters per section.
- Each explanation MUST contain exactly two sections:

1. "Justification and evidence of the smell:"

- Present evidence of incomplete or vague documentation.
- Use bullet points with the exact format:
- [Complete sentence]

2. "Suggested actions to address the smell:"

- Provide concrete, actionable recommendations to improve documentation completeness.
- Use a TABLE format with the following column:
- Action (must follow the exact format: "[LAZY] - [action title]")

Mandatory Output Structure:

- Use only the specified bullet format for evidence.
- Return a VALID JSON object containing the explanations for each affected endpoint.
- Do NOT include any text outside the JSON structure.

input_variables=["lazy_examples", "openapi_json"]

B Example of OpenAPI JSON

Box B.1: OpenAPI Specification Excerpt Used for Analysis

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "Order Management API",
    "description": "API for managing customer orders in an e-commerce platform",
    "version": "1.2.0"
  },
  "paths": {
    "/orders/createNewOrder": {
      "get": {
        "summary": "Create order",
        "operationId": "createNewOrder",
        "requestBody": {
          "required": true,
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/CreateOrderRequest"
              }
            }
          }
        },
        "responses": {
          "200": {
            "description": "Success",
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/GenericResponse"
                }
              }
            }
          }
        }
      }
    }
  },
  "components": {
    "schemas": {
      "CreateOrderRequest": {
        "type": "object",
        "properties": {
          "customer_id": { "type": "string" },
          "email": { "type": "string" },
          "phone": { "type": "string" },
          "payment_details": { "type": "object",
            "properties": {
              "card_number": {
                "type": "string",
                "example": "4111111111111111"
              },
              "card_holder": { "type": "string" }
            }
          }
        },
        "required": ["customer_id", "email"]
      },
      "GenericResponse": {
        "type": "object",
        "properties": {
          "status": {
            "type": "string",
            "example": "success"
          },
          "data": {
            "type": "object"
          }
        }
      }
    }
  }
}
```

C Example of An Analysis Report

Box B.2: Example of a Hermes Diagnostic Report

```
## API Info
- Title: "Order Management API"

## Endpoint Info
- Method: "GET"
- Path: "/orders/createNewOrder"

## Model
- gpt-oss:120b

## Identified Smells
Lazy, Security, Input, Response, Path & Method

### Explanations

### Lazy
- The summary is vague and the endpoint has no description, providing insufficient guidance about purpose, behavior, or usage.

### Security
- No authentication or authorization requirements are documented, and no security scheme is defined in the specification.

### Input
- Input fields lack semantic clarification and validation constraints such as format, pattern, or complete required declarations.

### Response
- The endpoint uses HTTP 200 for resource creation and provides only a generic response schema, without documented error cases.

### Path & Method
- The path is action-oriented ("createNewOrder"), and GET is used for an operation that semantically represents resource creation.

## Improvement Suggestions

[LAZY] - Improve documentation |
Provide a complete description including endpoint purpose, expected inputs, and outputs, with usage examples.

[SECURITY] - Define authentication |
Add Bearer token requirements and document the corresponding security scheme.

[INPUT] - Add validation |
Specify formats (e.g., UUID, email), validation constraints, and complete required field declarations.

[RESPONSE] - Improve responses |
Use 201 Created for successful creation and define detailed success and error response schemas.

[PATH_AND_METHOD] - Fix REST design |
Replace GET /orders/createNewOrder with POST /orders and adopt a resource-oriented path.
```